

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І. І. МЕЧНИКОВА
Факультет математики, фізики та інформаційних технологій
Кафедра математичного забезпечення комп'ютерних систем

ЗВІТ
до практичного завдання №2
з дисципліни «Мультиагентні системи»

Тема: «Аналіз бібліотеки PettingZoo та запуск середовища TicTacToe»

Виконав:

студент 4 курсу

спеціальності 126

Владислав Краковський

Перевірив:

Пенко В. Г.

Зміст

1. МЕТА РОБОТИ.....	3
2. ПОСТАНОВКА ЗАДАЧІ.....	4
3. ТЕОРЕТИЧНІ ВІДОМОСТІ ПРО PETTINGZOO.....	5
4. АНАЛІЗ ГРУПІ СЕРЕДОВИЩ PETTINGZOO.....	7
5. ОБРАНЕ СЕРЕДОВИЩЕ ТІСТАСТОЕ.....	8
6. ОПИС ПРОГРАМНОЇ РЕАЛІЗАЦІЇ.....	10
7. ІНСТРУКЦІЯ ЗАПУСКУ ПРОГРАМИ.....	11
8. РЕЗУЛЬТАТИ РОБОТИ ПРОГРАМИ.....	12
9. ТЕСТУВАННЯ.....	13
10. ВИСНОВКИ.....	14
ДОДАТОК А. ТЕКСТ ПРОГРАМИ.....	15

1. МЕТА РОБОТИ

Метою роботи є ознайомлення з бібліотекою PettingZoo як інструментом для моделювання мультиагентних середовищ, аналіз її основних можливостей, вибір одного середовища та проведення простого експерименту з агентами, що діють випадково.

Практична частина роботи полягає у запуску середовища TicTacToe, реалізації вибору допустимих ходів за допомогою маски дій, зборі статистики за декількома епізодами та підготовці програми для запуску як у консольному, так і у візуальному режимі.

2. ПОСТАНОВКА ЗАДАЧІ

У межах другого завдання необхідно виконати такі дії:

- ознайомитися з призначенням бібліотеки PettingZoo;
- розглянути основні принципи роботи АЕС API та Parallel API;
- проаналізувати групи середовищ, доступні у PettingZoo;
- обрати одне середовище для практичного експерименту;
- реалізувати запуск моделі з двома агентами;
- організувати вибір лише допустимих дій;
- зібрати статистику за серією запусків;
- передбачити можливість візуального відображення однієї гри;
- оформити результати у вигляді звіту.

3. ТЕОРЕТИЧНІ ВІДОМОСТІ ПРО PETTINGZOO

PettingZoo - це бібліотека Python, призначена для роботи з мультиагентними середовищами навчання з підкріпленням. Її можна розглядати як спеціалізований аналог Gymnasium для задач, у яких одночасно або по чергову діє декілька агентів.

У мультиагентних системах поведінка одного агента впливає на спостереження, винагороди та можливі дії інших агентів. Тому середовище має зберігати інформацію не тільки про один стан, а й про набір агентів, їхні ідентифікатори, спостереження, дії та винагороди.

Основною моделлю PettingZoo є AEC API - Agent Environment Cycle. У такій моделі агенти виконують дії послідовно: середовище надає поточному агенту спостереження, агент обирає дію, після цього середовище оновлює стан і передає керування наступному агенту.

Цикл Agent Environment Cycle (AEC) у PettingZoo



Рисунок 1 - Узагальнений цикл роботи AEC API

Окрім AEC API, PettingZoo підтримує Parallel API. Цей підхід використовується для середовищ, у яких декілька агентів можуть робити крок

одночасно. У межах даної роботи обрано саме АЕС-підхід, оскільки TicTacToe є покроковою грою.

4. АНАЛІЗ ГРУП СЕРЕДОВИЩ PETTINGZOO

PettingZoo містить декілька груп середовищ, які відрізняються типом задач, кількістю агентів, способом взаємодії та складністю моделювання.

Група	Призначення	Приклади задач
Classic	Класичні ігри та прості дискретні середовища.	TicTacToe, Chess, Rock-Paper-Scissors
Butterfly	Візуальні середовища з великою кількістю агентів та колективною поведінкою.	Pistonball, Cooperative Pong
MPE	Multi Particle Environments з агентами у двовимірному просторі.	simple_spread, simple_tag
Atari	Мультиагентні версії Atari-ігор.	Pong, Warlords
SISL	Середовища зі Stanford Intelligent Systems Laboratory.	Waterworld, Pursuit

Для практичного експерименту обрано групу Classic, оскільки вона містить зрозумілі покрокові ігри, які зручно пояснювати, запускати та перевіряти без складного навчання моделей.

5. ОБРАНЕ СЕРЕДОВИЩЕ TICTACTOE

Для експерименту було обрано середовище `tictactoe_v3`. У грі беруть участь два агенти: `player_1` та `player_2`. Вони по черзі обирають клітинки поля 3x3. Дії мають номери від 0 до 8, що відповідають дев'яти клітинкам поля.

Особливістю середовища є використання `action_mask`. Це маска допустимих дій, яка показує, які клітинки ще вільні. Якщо елемент маски дорівнює 1, хід дозволено; якщо 0 - хід у відповідну клітинку заборонено. Завдяки цьому агент не робить некоректних ходів.

Номер дії	Позиція на полі
0	верхній лівий кут
1	верхня центральна клітинка
2	верхній правий кут
3	середня ліва клітинка
4	центр
5	середня права клітинка
6	нижній лівий кут
7	нижня центральна клітинка
8	нижній правий кут



Рисунок 2 - Приклад стану ігрового поля TicTacToe

6. ОПИС ПРОГРАМНОЇ РЕАЛІЗАЦІЇ

Програму реалізовано мовою Python. Вона використовує бібліотеку PettingZoo для створення середовища TicTacToe, модуль argparse для обробки параметрів командного рядка, модуль random для випадкового вибору ходів та Counter для підрахунку результатів.

Основна логіка програми складається з трьох функціональних блоків: вибір допустимої дії, запуск одного епізоду та підсумковий аналіз результатів. Такий поділ дозволяє легко змінювати політику агентів або кількість запусків без переписування всієї програми.

Елемент програми	Призначення
choose_random_valid_action()	Отримує action_mask і випадково обирає один із дозволених ходів.
run_one_episode()	Створює середовище, запускає одну партію, накопичує нагороди та визначає результат.
print_results()	Підраховує перемоги, нічий, сумарні нагороди та середню кількість ходів.
main()	Зчитує параметри командного рядка та запускає необхідну кількість епізодів.

Агенти у цій роботі не навчаються. Їхня політика є випадковою, але не повністю хаотичною: кожен агент обирає випадкову дію тільки серед тих, які дозволені середовищем. Це робить експеримент коректним для демонстрації API PettingZoo.

7. ІНСТРУКЦІЯ ЗАПУСКУ ПРОГРАМИ

Для запуску програми необхідно встановити Python та бібліотеку PettingZoo з групою classic. Нижче наведено рекомендований порядок дій для Windows.

```
cd D:\MAS
python -m venv .venv
.venv\Scripts\activate
python -m pip install "pettingzoo[classic]"
```

Після встановлення залежностей можна запустити серію епізодів без графічного вікна:

```
python pettingzoo_tictactoe.py --episodes 100
```

Для візуальної демонстрації однієї гри використовується параметр --render:

```
python pettingzoo_tictactoe.py --episodes 1 --render
```

Для візуального режиму доцільно запускати саме один епізод. Якщо запускати багато партій з параметром --render, графічне вікно буде відкриватися багаторазово, що незручно для демонстрації та тестування.

8. РЕЗУЛЬТАТИ РОБОТИ ПРОГРАМИ

Після запуску без візуального режиму програма виводить кількість партій, політику агентів, кількість перемог кожного агента, кількість нічий, сумарні нагороди та середню кількість ходів за партію.

Наведений нижче приклад демонструє формат результату. Конкретні числові значення можуть змінюватися залежно від seed та кількості епізодів.

```
=== Результаты эксперимента PettingZoo TicTacToe ===  
  
Количество партий: 1  
Политика агентов: случайный выбор допустимого хода  
  
Победы и ничьи:  
player_1: 0  
player_2: 0  
ничья: 1  
  
Суммарные награды:  
player_1: 0  
player_2: 0  
  
Среднее количество ходов за партию: 9.0  
D:\University\study\4-year-2-semester\Мультиагентні с
```

Рисунок 3 - Приклад формату консольного виводу програми

У візуальному режимі PettingZoo відкриває окреме графічне вікно з ігровим полем. Такий режим зручний для демонстрації роботи середовища, але для статистики краще використовувати запуск без параметра `--render`, оскільки він працює швидше.

9. ТЕСТУВАННЯ

Для перевірки роботи програми було сформовано декілька тестових сценаріїв.

№	Команда	Очікуваний результат
1	<code>python pettingzoo_tictactoe.py -- episodes 1</code>	Запускається одна партія без графічного вікна, виводиться статистика.
2	<code>python pettingzoo_tictactoe.py -- episodes 100</code>	Запускається серія зі 100 партій, виводяться сумарні результати.
3	<code>python pettingzoo_tictactoe.py -- episodes 1 --render</code>	Відкривається графічне вікно TicTacToe для демонстрації гри.
4	<code>python pettingzoo_tictactoe.py -- episodes 10 --seed 123</code>	Результат відтворюється при повторному запуску з тим самим seed.

Програма не дозволяє агентам робити заборонені ходи, оскільки дія обирається лише з індексів, для яких `action_mask` дорівнює 1. Це є важливим елементом коректної роботи з середовищем TicTacToe.

10. ВИСНОВКИ

У роботі було розглянуто бібліотеку PettingZoo, її призначення та основні підходи до організації мультиагентних середовищ. Особливу увагу приділено АЕС API, який добре підходить для покрокових ігор, де агенти діють по черзі.

Для практичного експерименту було обрано середовище TicTacToe. Реалізована програма створює середовище, запускає серію партій, використовує `action_mask` для вибору коректних дій, накопичує статистику та підтримує візуальне відображення однієї гри.

Отримана програма демонструє базовий підхід до роботи з PettingZoo: створення середовища, взаємодію агентів з ним, обробку спостережень, вибір дій, аналіз винагород і підбиття підсумків після серії епізодів.

ДОДАТОК А. ТЕКСТ ПРОГРАМИ

Нижче наведено текст програми `pettingzoo_tictactoe.py`, яка використовується для запуску експерименту в середовищі `PettingZoo TicTacToe`.

```
"""
Задание 2. PettingZoo: запуск и анализ мультиагентной среды TicTacToe.

Что делает программа:
1. Создаёт среду TicTacToe из библиотеки PettingZoo.
2. Запускает несколько партий между двумя случайными агентами.
3. В каждой партии агенты выбирают только допустимые ходы.
4. Собирает статистику: победы player_1, победы player_2, ничьи, суммарные награды.
5. При необходимости показывает одну партию визуально через render_mode="human".

python -m pip install "pettingzoo[classic]" - установить библиотеку

Запуск без визуализации:
python pettingzoo_tictactoe.py --episodes 100

Запуск одной визуальной партии:
python pettingzoo_tictactoe.py --episodes 1 --render
"""

# argparse нужен для чтения параметров из командной строки:
# --episodes, --render, --seed.
import argparse

# random нужен для случайного выбора хода агентом.
import random

# Counter нужен для удобного подсчёта побед, поражений и ничьих.
from collections import Counter

# Импортируем классическую среду TicTacToe из PettingZoo.
# Это готовая мультиагентная среда крестиков-ноликов.
from pettingzoo.classic import tictactoe_v3

def choose_random_valid_action(observation, rng):
    """
    Выбирает случайное допустимое действие для текущего агента.

    В PettingZoo TicTacToe наблюдение observation является словарём.
    В нём есть поле "action_mask".

    action_mask – это список из 9 элементов:
    - 1 означает, что клетка свободна и туда можно ходить;
    - 0 означает, что клетка занята и ход туда запрещён.

    Позиции соответствуют клеткам поля 3x3:

        0 | 1 | 2
        -----
        3 | 4 | 5
        -----
        6 | 7 | 8
    """

    # Получаем маску допустимых действий.
    action_mask = observation["action_mask"]
```

```

# Создаём список всех действий, которые разрешены.
valid_actions = []

# enumerate даёт одновременно номер действия и значение в маске.
for action_id, is_allowed in enumerate(action_mask):
    # Если is_allowed == 1, значит действие разрешено.
    if is_allowed == 1:
        valid_actions.append(action_id)

# Случайно выбираем одно действие из списка допустимых.
return rng.choice(valid_actions)

def run_one_episode(episode_id, seed=42, render=False):
    """
    Запускает одну партию TicTacToe.

    episode_id:
        номер эпизода. Используется для изменения seed,
        чтобы партии не были одинаковыми.

    seed:
        базовое зерно генератора случайных чисел.

    render:
        если True, PettingZoo откроет визуальное окно игры.
        Для статистики render лучше держать False.
    """

    # Создаём отдельный генератор случайных чисел.
    # Так результаты можно воспроизводить при одинаковом seed.
    rng = random.Random(seed + episode_id)

    # Создаём среду.
    # render_mode="human" открывает графическое окно.
    # render_mode=None запускает игру без окна, быстрее и стабильнее.
    env = tictactoe_v3.env(render_mode="human" if render else None)

    # Сбрасываем среду перед началом новой партии.
    # В новых версиях PettingZoo reset может принимать seed.
    # В старых версиях может не принимать, поэтому используем try/except.
    try:
        env.reset(seed=seed + episode_id)
    except TypeError:
        env.reset()

    # Словарь для суммарных наград игроков за всю партию.
    total_rewards = {
        "player_1": 0,
        "player_2": 0
    }

    # Счётчик ходов.
    moves_count = 0

    # agent_iter() возвращает агентов по очереди.
    # В TicTacToe сначала ходит один агент, потом другой, и так далее.
    for agent in env.agent_iter():
        # env.last() возвращает данные для текущего активного агента:
        # observation - наблюдение текущего агента;
        # reward - награда текущего агента за предыдущий переход;
        # termination - естественное завершение игры;
        # truncation - принудительное завершение, например по лимиту;

```

```

# info          - дополнительная служебная информация.
observation, reward, termination, truncation, info = env.last()

# Добавляем награду текущему агенту.
total_rewards[agent] += reward

# Если игра уже завершилась, действие должно быть None.
# Это требование API PettingZoo.
if termination or truncation:
    action = None
else:
    # Иначе выбираем случайный допустимый ход.
    action = choose_random_valid_action(observation, rng)

    # Увеличиваем счётчик реальных ходов.
    moves_count += 1

# Передаём действие в среду.
# После этого среда переключится на следующего агента.
env.step(action)

# Закрываем среду после завершения партии.
# Это особенно важно при render_mode="human", чтобы закрыть окно.
env.close()

# Определяем победителя по суммарной награде.
# В TicTacToe победитель получает большую итоговую награду.
if total_rewards["player_1"] > total_rewards["player_2"]:
    winner = "player_1"
elif total_rewards["player_2"] > total_rewards["player_1"]:
    winner = "player_2"
else:
    winner = "draw"

# Возвращаем результат партии в виде словаря.
return {
    "episode_id": episode_id,
    "winner": winner,
    "player_1_reward": total_rewards["player_1"],
    "player_2_reward": total_rewards["player_2"],
    "moves_count": moves_count
}

def print_results(results):
    """
    Выводит общую статистику по всем запущенным партиям.
    """

    # Считаем количество побед каждого результата.
    winner_counter = Counter(result["winner"] for result in results)

    # Считаем суммарные награды игроков.
    player_1_total = sum(result["player_1_reward"] for result in results)
    player_2_total = sum(result["player_2_reward"] for result in results)

    # Считаем среднее число ходов.
    average_moves = sum(result["moves_count"] for result in results) / len(results)

    print("\n=== Результаты эксперимента PettingZoo TicTacToe ===\n")

    print("Количество партий:", len(results))
    print("Политика агентов: случайный выбор допустимого хода")
    print()

```



```

print("Победы и ничьи:")
print("player_1:", winner_counter.get("player_1", 0))
print("player_2:", winner_counter.get("player_2", 0))
print("ничья:", winner_counter.get("draw", 0))
print()

print("Суммарные награды:")
print("player_1:", player_1_total)
print("player_2:", player_2_total)
print()

print("Среднее количество ходов за партию:", round(average_moves, 2))

def main():
    """
    Главная функция программы.

    Она:
    1. Читает параметры командной строки.
    2. Запускает нужное количество партий.
    3. Выводит статистику.
    """

    # Создаём объект парсера аргументов командной строки.
    parser = argparse.ArgumentParser(
        description="PettingZoo TicTacToe: случайные агенты"
    )

    # --episodes задаёт количество партий.
    parser.add_argument(
        "--episodes",
        type=int,
        default=100,
        help="Количество партий для запуска"
    )

    # --seed задаёт базовое зерно случайности.
    parser.add_argument(
        "--seed",
        type=int,
        default=42,
        help="Зерно генератора случайных чисел"
    )

    # --render включает визуальное отображение.
    parser.add_argument(
        "--render",
        action="store_true",
        help="Показать визуальное окно игры"
    )

    # Читаем аргументы.
    args = parser.parse_args()

    # Если включён render и пользователь просит много партий,
    # предупреждаем, что лучше показывать только одну.
    # Иначе окна будут открываться/закрываться много раз. Это не смерть, но близко.
    if args.render and args.episodes > 1:
        print("Включён --render. Для визуального режима лучше запускать 1 партию.")
        print("Программа всё равно продолжит выполнение, но это может быть неудобно.\n")

    # Список для хранения результатов всех партий.
    results = []

```

```

# Запускаем партии.
for episode_id in range(args.episodes):
    result = run_one_episode(
        episode_id=episode_id,
        seed=args.seed,
        render=args.render
    )

    results.append(result)

# Выводим итоговую статистику.
print_results(results)

# Эта проверка означает:
# запускать main() только если файл запущен напрямую,
# а не импортирован как модуль.
if __name__ == "__main__":
    main()

```